Algorithms 3

Mingxun Zhou (mingxunz@ust.hk)

COMP2711 Discrete Mathematical Tools

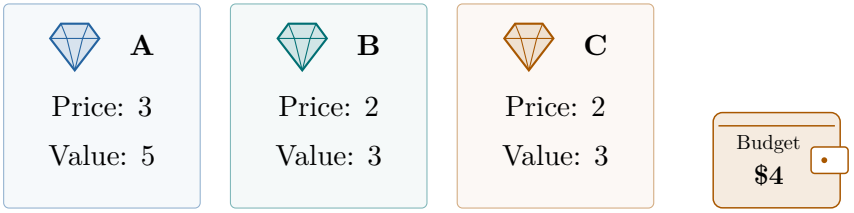
Learning objectives

After this lecture, you should be able to:

1. **Brute force.** Solve knapsack by checking all subsets and count the work.
2. **Greedy and approximation.** Compare greedy rules and prove Carol's half-optimum guarantee.
3. **P, NP, and NP-completeness.** Distinguish solving from verifying, and explain polynomial-time reductions.

1 Knapsack and brute force

You have a budget of 4. Treasure A costs 3 and has value 5; B and C each cost 2 and have value 3. A is the most valuable single item, but B and C together fit and give value 6.



1.1 The 0/1 knapsack problem

There are n distinct items, each with positive integer price p_i dollars and value v_i points. The budget is a nonnegative integer B. Select a subset S maximizing

$$V(S) = \sum_{i \in S} v_i \qquad \text{subject to} \qquad \sum_{i \in S} p_i \leq B.$$

Each item is taken whole or not taken, and there is one copy of each item. The empty subset is allowed. Write Best for the maximum feasible value. Discard items costing more than B, since they cannot be selected.

MUST UNDERSTAND Brute force

A brute-force algorithm checks the candidates directly. For knapsack, enumerate all 2^n subsets, test their total price, and retain the feasible subset with greatest value [1, Section 3.3].

Subset	Price	Value	Feasible for $B = 4$?
$\varnothing$	0	0	Yes
A	3	5	Yes
B	2	3	Yes
C	2	3	Yes
A,B	5	8	No
A,C	5	8	No
B,C	4	6	Yes
A,B,C	7	11	No

Correctness: Every feasible subset is checked.

$$\text{Time} = 2^n \text{ subsets} \times \Theta(n) \text{ work per subset} = \Theta(n2^n).$$

Store only the current and best subsets: $O(n)$ extra space.

2 Greedy algorithms and approximation

MUST UNDERSTAND Greedy algorithm

A greedy algorithm repeatedly commits to a locally preferred choice. Its correctness or quality guarantee must be proved for the particular rule and problem.

2.1 Three strategies

Each strategy scans its sorted list and takes an item if it fits; otherwise it skips the item and continues.

Strategy	Rule
Alice	Sort by decreasing value v_i.
Bob	Sort by decreasing value per dollar v_i/p_i (性价比).
Carol	Run both strategies and keep the selection with greater value.

Sorting costs $O(n \log n)$ comparisons; the scans take $O(n)$ arithmetic work. Compare ratios exactly by cross-products $v_i p_j$ and $v_j p_i$.

Alice can be arbitrarily bad. Let $B = 10k$, $k \geq 2$.

Item type	Number	Price each	Value each
Large	1	$10k$	100
Small	k	10	99

Alice takes the large item; the optimum takes all small items:

$$V_A = 100, \qquad \text{Best} = 99k, \qquad \frac{V_A}{\text{Best}} = \frac{100}{99k} \longrightarrow 0.$$

For $k = 10$: 100 versus 990.

Bob can also be arbitrarily bad. Let $B = M > 2$.

Item	Price	Value	Value/price
Small	1	2	2
Large	M	M	1

Bob takes the small item; the large item no longer fits. Thus $V_B/\text{Best} = 2/M \to 0$. Alice and Carol take the large item, giving M.

A small example for Bob. Add item D of price 1 and value 2 to the original treasures. The ratio order is D,A,B,C. For budget 6, Bob takes D,A,B and gets value 10, which is optimal. For budget 5, he takes D,A and gets 7, but D,B,C gives 8.

MUST UNDERSTAND Approximation guarantee

For a maximization problem, an α-approximation returns a feasible solution with value at least $\alpha\,\text{Best}$ on every input, where $0 < \alpha \leq 1$.

Theorem 2.1. *Carol's strategy satisfies*

$$\tfrac{1}{2}\text{Best} \leq V_C \leq \text{Best}.$$

Carol guarantees at least half the optimum on every input. She need not be exact: with budget 4, both scans choose A (value 5), but B,C gives 6.

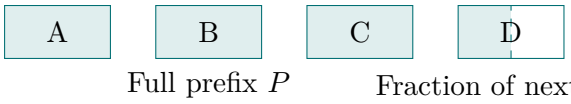
2.2 Proof of the half-optimum guarantee

Allow a fraction $x_i \in [0, 1]$ of item i, with price $x_i p_i$ and value $x_i v_i$. Every whole-item selection remains feasible, so

$$\text{Best} \leq \text{Best}_{\text{frac}}.$$

For example, taking A and half of B has price 4 and value $5 + 3/2 = 6.5$.

Why fractional greedy is optimal. Fill in decreasing ratio order. Any other selection can be improved by moving spending from a lower-ratio item to an unfilled higher-ratio item. Repeating this exchange, and spending the remaining budget, gives full items followed by at most one partial item [2].



Compare with Alice and Bob. Let j be Bob's first skipped item and P the items before it.

- Bob keeps P, so $V(P) \le V_B$.
- Fractional greedy takes P and a fraction $\theta \in [0, 1)$ of j.

Thus

$$\text{Best} \le \text{Best}_{\text{frac}} = V(P) + \theta v_j \le V_B + v_j.$$

Item j fits by itself. Alice takes a highest-value feasible item first, so $v_j \le V_A$. Consequently,

$$\text{Best} \le V_A + V_B \le 2\max(V_A, V_B) = 2V_C.$$

Since Carol's selection is feasible, also $V_C \le \text{Best}$. This proves the theorem. If all retained items fit together, both scans are optimal; if none remain, the empty selection is optimal.

Exact knapsack also has a dynamic-programming algorithm, covered in a later algorithm class. Its budget-dependent running time is not polynomial in the binary input length.

3 P, NP, and NP-completeness

3.1 Finding a selection and checking one

The optimization problem asks for the maximum value. **Decision knapsack** receives a target T and asks whether some subset S satisfies

$$\sum_{i \in S} p_i \le B \qquad \text{and} \qquad \sum_{i \in S} v_i \ge T.$$

All numbers are encoded in binary. With the original treasures, $B = 4, T = 6$ is a Yes instance: B,C is a valid selection. With $T = 7$, the answer is No.

A **certificate** lists the selected items as an n-bit vector. Check their total price and value. Here 011 (B,C) certifies Yes; 100 (A) fails. One failed proposal does not rule out every selection.

MUST UNDERSTAND Polynomial time and input length

An algorithm is polynomial time if its worst-case time is $O(L^c)$ for some fixed constant c, where L is the encoded input length. An integer $N \ge 1$ uses $\lfloor \log_2 N \rfloor + 1$ bits: a loop with N iterations can be exponential in its encoding length.

Verification uses $O(n)$ additions and comparisons. For B-bit input numbers, sums need only $O(B + \log n)$ bits, so verification takes polynomial time.

MUST UNDERSTAND P and NP

P Decision problems solvable by deterministic polynomial-time algorithms.

NP
Decision problems whose Yes instances have polynomial-length certificates checkable by deterministic polynomial-time algorithms. Every Yes instance has an accepted certificate; no No instance has one.

Decision knapsack is in NP by the verifier above. NP stands for nondeterministic polynomial time. Every problem in P belongs to NP: decide the instance directly and ignore the certificate. Thus $\mathrm{P} \subseteq \mathrm{NP}$.

The question $\mathrm{P} \stackrel{?}{=} \mathrm{NP}$ asks whether polynomial-time verification always implies polynomial-time decision. It remains open [3]. An exponential-time brute-force algorithm or a failed greedy rule does not establish that a problem is outside P.

3.2 Reductions and NP-completeness

MUST UNDERSTAND Polynomial-time reduction

A reduction $A \leq_p B$ is a polynomial-time transformation h satisfying

$$x \text{ is Yes for A} \quad \Longleftrightarrow \quad h(x) \text{ is Yes for B.}$$

It preserves both Yes and No answers. An algorithm for B can solve A by transforming the input and then applying that algorithm.

If reduction time is $p(L)$, output length is at most $q(L)$, and B has nondecreasing time bound T, then

$$\text{Time for A} \leq \underbrace{p(L)}_{\text{transform}} + \underbrace{T(q(L))}_{\text{solve B}},$$

where p, q are polynomials.

MUST UNDERSTAND NP-complete

A decision problem B is **NP-complete** if:

1. B belongs to NP.
2. Every problem in NP reduces to B in polynomial time.

A polynomial-time algorithm for one NP-complete problem would imply $\mathrm{P} = \mathrm{NP}$.

3.3 3SAT and knapsack

A **literal** is a Boolean variable or its negation. A 3-CNF formula is an AND of clauses, each an OR of three literals. 3SAT asks whether some assignment makes the formula true. For

$$(x \vee y \vee z) \wedge (\neg x \vee y \vee z) \wedge (x \vee \neg y \vee \neg z),$$

$x = \mathrm{T}, y = \mathrm{F}, z = \mathrm{T}$ satisfies every clause. The all-false assignment fails the first. An assignment is a certificate; checking all clauses takes polynomial time, so 3SAT is in NP.

Theorem 3.1 (Stated without proof)**.** *3SAT and decision 0/1 knapsack with binary-encoded positive integer prices and values are NP-complete.*

A standard proof uses reductions from 3SAT to subset sum and from subset sum to knapsack. We demonstrate the second step [4].

Subset sum to knapsack. Given positive integers $a_1, \ldots, a_n$ and a target t, ask whether some subset sums exactly to t. Set $p_i = v_i = a_i$ and $B = T = t$. The knapsack conditions become

$$\sum_{i \in S} a_i \leq t \quad \text{and} \quad \sum_{i \in S} a_i \geq t,$$

which are equivalent to equality. Copying these numbers takes polynomial time. For $[1, 2, 3, 4]$ and target 5, selecting items $1, 4$ answers Yes to both problems.

Because 3SAT and decision knapsack are both NP-complete, each reduces to the other, and

$$\text{3SAT} \in \text{P} \iff \text{Decision Knapsack} \in \text{P} \iff \text{P} = \text{NP}.$$

NP-completeness does not prove an exponential lower bound. The label applies here to decision knapsack; the optimization problem is NP-hard.

3.4 Why approximation does not decide every instance

Carol's returned value gives $V_C \leq \text{Best} \leq 2V_C$. For target T:

Condition	Conclusion
$V_C \geq T$	Yes: the returned selection meets the target.
$2V_C < T$	No: even the upper bound misses the target.
$V_C < T \leq 2V_C$	The guarantee alone does not decide.

For the original treasures, $V_C = 5$ gives $5 \leq \text{Best} \leq 10$. This does not decide target 6, even though B,C supplies a Yes certificate. Approximation provides useful bounds without always finding the exact answer.

4 Optional review: repeated squaring

To compute $a^b \bmod m$, with $b \geq 0, m \geq 2$, use the binary digits of the exponent:

$r \leftarrow 1$; $\quad s \leftarrow a \bmod m$; $\quad e \leftarrow b$
while $e > 0$:
  if e is odd: $r \leftarrow rs \bmod m$
  $s \leftarrow s^2 \bmod m$; $\quad e \leftarrow \lfloor e/2 \rfloor$
return r

Correctness. Maintain $rs^e \equiv a^b \pmod m$. Writing $e = 2q + d$, $d \in \{0, 1\}$, one update preserves the value:

$$(rs^d)(s^2)^q = rs^{2q+d} = rs^e.$$

At termination, $e = 0$, so $r \equiv a^b \pmod{m}$.

Cost. At most two modular multiplications per bit of b: $O(\log b)$ for $b \geq 1$. For $b = 0$, the cost is constant.

Example. Powers $3^1, 3^2, 3^4, 3^8$ have residues $3, 2, 4, 2$ modulo 7.

$$3^{13} \equiv 2 \times 4 \times 3 \equiv 3, \qquad 3^{11} \equiv 2 \times 2 \times 3 \equiv 5 \pmod{7}.$$

5 Exercises

1. In the original three-treasure instance, find the optimum for budgets 5 and 1. How many feasible subsets are there for budget 4?
2. A half-optimum algorithm returns value 12. Bound Best. Which targets among $10, 20, 25$ can its guarantee decide?
3. Why does Alice's failure example rule out every fixed positive approximation guarantee?
4. Explain why the first item Bob skips fits by itself in the approximation proof.
5. One proposed selection fails the target. Does that imply a No instance?
6. Suppose decision knapsack is in P. What follows for 3SAT and for P versus NP?
7. Verify the subset-sum reduction in both directions.
8. Compute 3^{11} mod 7 by repeated squaring.

Answers

1. At budget 5, take A,B or A,C for value 8. At budget 1, only the empty subset fits, with value 0. Budget 4 has five feasible subsets, including the empty one.
2. $12 \leq$ Best ≤ 24. Target 10 is Yes; target 25 is No; target 20 is unresolved by these bounds.
3. The ratio is $100/(99k)$, which can be made smaller than any fixed positive constant by increasing k.
4. Items costing more than the budget were discarded first. Bob skips this item because of the money already spent, not because it exceeds the total budget.
5. No. A different selection may succeed. A No answer rules out every selection.
6. 3SAT is in P, and P = NP, since every NP problem reduces to decision knapsack in polynomial time.
7. A subset summing to t has price t and value t. Conversely, feasible price at most t and value at least t force equality because prices and values coincide.
8. The selected residues for exponents $8, 2, 1$ are $2, 2, 3$, so the answer is $12 \bmod 7 = 5$.

References

[1] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill Education, 8th edition, 2019.

[2] Cornell University. Knapsack approximation: Cs 4820 lecture notes. https://www.cs.cornell.edu/courses/cs4820/2019fa/lectures/approx-knapsack.pdf, 2019.

[3] Clay Mathematics Institute. P vs np. https://www.claymath.org/millennium/p-vs-np/.

[4] David Mix Barrington. Cs 611 lecture 18: Subset sum and knapsack reductions. https://people.cs.umass.edu/~barring/cs611/lecture/18.pdf.